

# Übungen

## Übung 1 - Data Provider

### Vorbedingungen

Sie haben den Branch "lessons/04-asynchronous-processes" ausgecheckt oder das erste Kapitel der Übungen erfolgreich abgeschlossen.

### Motivation

Lazy Loading verstehen

### Zeit

90 Minuten

### Aufgabe 1

Erstellen Sie die Klasse `FilterableDataProvider<T>` (gefiltert wird später) im Service Modul und fügen Sie 2 Methoden hinzu:

- `Stream<T> fetch(int offset, int limit)`
- `long count()`

Erstellen sie eine Methode in der Klasse `TaskService`, die einen `FilterableDataProvider` für `Task` Objekte zurückliefert. Erstellen Sie dafür falls nötig weitere Repository-Methoden.

Reichen Sie nun den Data Provider anstelle der `Task`-Liste an die UI weiter und transferieren Sie Ihren `DataProvider` in einen Vaadin `DataProvider` und setzen Sie ihn mittels `setItems()` in das Grid.

### Aufgabe 2

Erweitern Sie den `FilterableDataProvider` um eine Sortierfunktion. Erstellen Sie dafür eine eigene Klasse `SortOrder`, die hilft, die Sortierinformationen von Vaadin nach Spring zu transportieren. Sie sollte zwei Eigenschaften besitzen:

- `property` - String
- `direction` - Enum (ASC, DESC)

Fügen Sie nun eine Liste mit Elementen der neuen Klasse `SortOrder` der Methode `fetch` des `FilterableDataProvider` hinzu. Im UI Modul können nun die Vaadin Sortierinformationen (`Query#getSortOrders()`) in unsere neue `SortOrder` Liste umgewandelt werden. Im Persistenz-Modul müssen diese Informationen dann entweder in

die Spring Sortierform konvertier, oder in eine Query eingebaut werden. Damit das Grid nun sortierfähig wird, erweitern Sie nun die Spalten des Grids, indem Sie deren Erzeugung mit `.setSortProperty(String)` vervollständigen und die entsprechenden Felder der Klasse `Task` dort angeben. Das Vaadin Grid wird diese dann automatisch in die Query einfügen.

## Aufgabe 3

Erweitern sie den `FilterableDataProvider` indem Sie eine Filtermöglichkeit hinzufügen. Erstellen Sie hierzu eine Klasse `TaskFilter` mit 3 Eigenschaften:

- `search` - `String`
- `min` - `LocalDate`
- `max` - `LocalDate`

Fügen Sie die Klasse `TaskFilter` der methode `count` und `fetch` Ihres `FilterableProvider` hinzu und benutzen Sie diese im Persistenz Modul um die Query zu verfeinern. Um diese Filtermöglichkeit nutzen zu können, müssen wir in der UI entsprechende Filterelemente platzieren. Als einfaches Beispiel, können wir unserem Grid einen zweiten Header spendieren und dort ein Textfeld für die "Description" und zwei `DatePicker` für das "Due Date" einfügen. Fügen Sie jeder Such-Komponente einen `ValueChangeListener` hinzu, um die Filtereingabe anzustoßen. In diesem Listener muss der Filter unseres `FilterableDataProviders` gesetzt werden und anschließend der Grid `DataProvider` mittels `taskGrid.getDataProvider().refreshAll()` zurückgesetzt werden.

# Übung 2 - Komplexe Komponenten & Binder

## Vorbedingungen

Übung 1 ist abgeschlossen oder Sie haben den Branch "lessons/05-data-providers" ausgecheckt.

## Motivation

Komplexe Komponenten und Binder verstehen

## Abkürzung

Der Branch "lessons/05-binder-prepared" ist ausgecheckt. Es geht weiter mit Aufgabe 3.

## Zeit

120 Minuten

## Aufgabe 1

Erstellen Sie eine Bean und ein Repository für Die Folgende Struktur:

- Person
  - PhoneNumber
    - Country Code
    - Area Code
    - Number
  - Address
    - Address 1
    - Address 2
    - Zip Code
    - City
    - Country

Bitte beachten Sie, dass Persistenzklassen nicht in höheren Schichten verwendet werden sollten. Um das zu ermöglichen, kann man entweder dem Repository eine `create()` Methode geben oder über `add(...)` Methoden das Konstruieren der Objekte kapseln. Z.B.: `Person#addAddress(String address1, ...)`.

## Aufgabe 2

Erstellen Sie eine View mit einem Grid zum Verwalten der Kontakte. Bei Auswahl der Person (Single Select) soll rechts neben dem Grid ein Editor angezeigt werden.

## Aufgabe 3

Implementieren Sie den Editor. Er soll 3 Bereiche besitzen.

1. Änderung des Namens der Person
2. Änderung der Telefonnummer
  - Komplexe Komponente mit `HasValue<E, PhoneNumber>`
3. Änderung der Adresse
  - Komplexe Komponente mit `HasValue<E, Address>`

Unter dem Formular befinden sich 2 Buttons. Einer zum Zurücksetzen und einer zum Speichern.